Software Architecture and Design Specification - Hostel Management System

1. Introduction

1.1 Purpose

This document describes the architecture and design of the Hostel Management System (HMS). It explains how the functional and non-functional requirements written in the SRS will be built using Python and Django: which components the system has, how they talk to each other, how data is stored, and how the important flows such as room allocation and check-out work step by step. It is the main reference for the team while coding and for the testers while writing test cases.

1.2 Scope

This document covers all functions listed in the SRS:

- Login, logout and role-based access for Students and Hostel Administrators.
- Student records: registration, update, profile view and search.
- Hostel and room management, including room capacity and live availability.
- Room allocation with over-allocation prevention, plus modify, cancel and allocation history.
- Student check-in and check-out.
- A basic administrative summary dashboard.

Online fee payment, biometric attendance, visitor management and integration with other college systems are out of scope, same as in the SRS.

1.3 Audience

Project developers, testers, the course coordinator and evaluators, and anyone who maintains the system later.

1.4 Definitions

Term	**Meaning**
HMS	Hostel Management System
SRS / SAD / STP	Software Requirements Specification / Software Architecture and Design document (this document) / Software Test Plan
RTM	Requirements Traceability Matrix
MVT	Model View Template, the pattern Django uses (similar to MVC)
ORM	Object Relational Mapper. Django ORM converts Python model classes into database

Term	Meaning
	tables and queries
App	A self-contained Django module (for example students, rooms, allocations)
ADR	Architecture Decision Record, a short note on an important design choice and why it was made
ACTIVE allocation	An allocation that is currently using a bed in a room (not cancelled and not checked out)
CSRF	Cross Site Request Forgery, an attack Django blocks using a hidden token in every form
RBAC	Role Based Access Control
STRIDE	Threat model categories: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege

2. Architecture

2.1 Goals & Constraints

Goals:

- **Correct allocation:** a room must never have more active allocations than its capacity (HMS-F-011, HMS-NF-006).
- **Data integrity:** availability shown on screen must always match the stored allocations.
- **Security:** only logged-in users can use the system, and students can only see their own data (HMS-SR-001 to HMS-SR-006).
- **Performance:** normal pages load within 3 seconds for at least 90% of requests (HMS-NF-001).
- **Maintainability:** each feature lives in its own Django app so one module can change without breaking others (HMS-NF-005).
- **Simplicity:** a design a team of four students can build, test and demo within the semester.

Constraints:

- Must be a web application built with Python and Django (course rule and SRS 2.5).
- SQLite is used during development; the deployed version may use any Django supported relational database.
- Team of four students, academic mini-project timeline.
- No external systems (payment gateway, college ERP, biometric devices) in this version.

2.2 Stakeholders & Concerns

Stakeholder	Main Concerns
Students	Easy to see their room and allocation status; their personal data stays private.
Hostel Administrator	Fast and correct allocation, no double booking, clear view of occupancy, easy check-in/check-out.

Stakeholder	Main Concerns
Project Administrator	Simple setup and deployment, access to Django admin for fixing data, backups.
Course Coordinator / Evaluators	Design follows the SRS, uses proper SE practice (UML, traceability, testing).
Developers (project team)	Clear module boundaries, low coupling, parallel work without merge conflicts.
Testers	Every requirement maps to a testable component and a clear expected behaviour.

2.3 Component (UML) Diagram

The component diagram already prepared by the team. It shows the three tiers (browser, Django application logic, database) and the interfaces each module provides (lollipop) and uses (socket). Figure 2 is the UML class diagram of the domain model, which shows the Django models behind these components.

Figure 1: HMS Component Diagram (socket and lollipop notation)

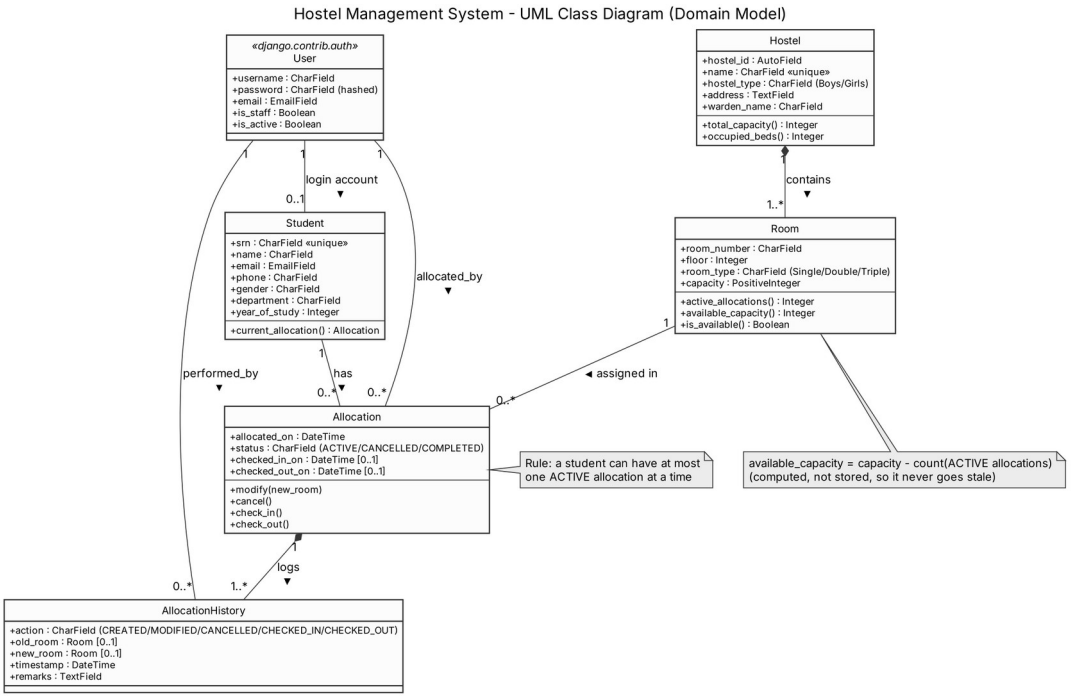


Figure 2: HMS UML Class Diagram (domain model / Django models)

2.4 Component Descriptions

Component	Django App	Responsibility	Provided Interface(s)	SRS Req.
Web Browser / UI Client	templates/, static/	Renders HTML pages built with Django templates and Bootstrap; sends HTTP(S) requests and forms.	-	3.1, HMS-NF-004

Component	Django App	Responsibility	Provided Interface(s)	SRS Req.
Authentication Module	accounts	Login, logout, sessions, password hashing (django.contrib.auth), role checks using is_staff and a custom @admin_required decorator.	IAuthService, ISession	F-001, F-002, SR-001 to SR-005
Student Module	students	Create, update, view and search student records; links a Student to its login User.	IStudentProfile	F-003, F-004, F-005, F-017
Hostel Module	hostels	Add and maintain hostel details (name, type, address, warden).	IHostelMgmt	F-006
Room Module	rooms	Add and update rooms inside a hostel; calculates available capacity of each room.	IRoomMgmt, IAvailability	F-007, F-008, F-009
Allocation Module	allocations	Allocate, modify and cancel rooms; enforces capacity rule inside a database transaction; writes allocation history.	IAllocation, IHistory	F-010 to F-014, F-018
Check-In Module	allocations (checkin views)	Records check-in date for a student who has an active allocation.	ICheckIn	F-015
Check-Out Module	allocations (checkout views)	Records check-out, closes the allocation and releases the bed.	ICheckOut	F-016
Dashboard Module	dashboard	Read-only summary: total students, rooms, beds, occupied and free beds per hostel.	ISummary	F-019
Django ORM Engine	(framework)	Maps models to tables, runs queries, migrations and transactions.	IDataAccess	NF-002, NF-006
Database	SQLite / RDBMS	Stores users, students, hostels, rooms, allocations and history.	SQL connection	NF-002

2.5 Chosen Architecture Pattern and Rationale

HMS uses a layered (3-tier) architecture, built as a modular monolith using Django's MVT pattern: one Django project split into self-contained apps, one per component in Figure 1.

- **Presentation layer:** Django templates (HTML, CSS, Bootstrap) shown in the browser.
- **Application / business layer:** Django views handle HTTP, login and role checks only; business rules (capacity check, allocation states) live in service functions inside each app, for example allocate_room() in allocations/services.py.
- **Data layer:** Django models, the ORM and the relational database.

Why this pattern: it matches how Django already works, keeps business rules like the capacity check in one place where they are easy to unit test, and splitting into apps lets each team member own a module. A single deployable unit is much easier to run, test and demo than many services.

Architecture Decision Records:

ADR	Decision	Alternatives considered	Rationale
ADR-01	Layered modular monolith in Django, one app per module, service functions separate from	Microservices; plain MVC with all logic inside views.	Microservices add network calls and deployment work that a four person team cannot justify for one hostel. A

ADR	Decision	Alternatives considered	Rationale
	views.		service layer keeps rules testable without HTTP.
ADR-02	Server-rendered Django templates with Bootstrap.	React SPA + Django REST Framework.	SRS only needs forms and tables; one codebase is faster to build and the team already knows Django templates.
ADR-03	SQLite for development, PostgreSQL or MySQL allowed for deployment.	PostgreSQL everywhere.	SQLite needs no setup on team laptops; using only the ORM (no raw SQL) keeps the database swappable.
ADR-04	Room availability is computed (capacity minus ACTIVE allocations), never stored as a counter.	A free_beds column updated on every allocate / cancel.	A stored counter can drift out of sync after a bug or a failed request; computing it keeps HMS-NF-006 true by design.
ADR-05	Django built-in auth and sessions, is_staff marks the Hostel Administrator role.	Custom user table and login code; JWT tokens.	Well tested, hashes passwords (SR-003) and manages sessions (SR-005) with no custom security code.
ADR-06	Allocations are never deleted; status becomes CANCELLED or COMPLETED and every change is written to AllocationHistory.	Delete the allocation row on cancel / check-out.	Needed for allocation history (F-018) and to know which admin made each change.

2.6 Technology Stack & Data Stores

Layer / Purpose	Technology
Language	Python 3.11 or later
Web framework	Django 5.x (MVT, forms, messages framework, admin site)
Frontend	Django Template Language, HTML5, CSS, Bootstrap 5
Database	SQLite 3 (development); PostgreSQL or MySQL (optional, deployment)
Data access	Django ORM and migrations
Authentication	django.contrib.auth (PBKDF2 password hashing, session cookies)
Web server (deployment)	Gunicorn behind Nginx with HTTPS
Testing	Django TestCase (unittest), Django test Client, coverage.py, Selenium (UI flows), Locust (load)
Version control	Git and GitHub (feature branches and pull requests)
Diagrams	PlantUML, draw.io

Figure 3 shows the deployment view: during development each member runs the Django development server with a local SQLite file, and the demo / deployed version runs behind Gunicorn and Nginx. Figure 4 is the entity-relationship view of the database (one table per model in Figure 2).

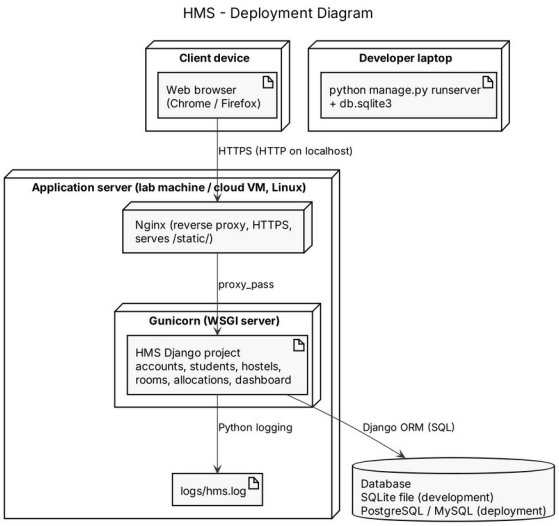


Figure 3: HMS deployment diagram

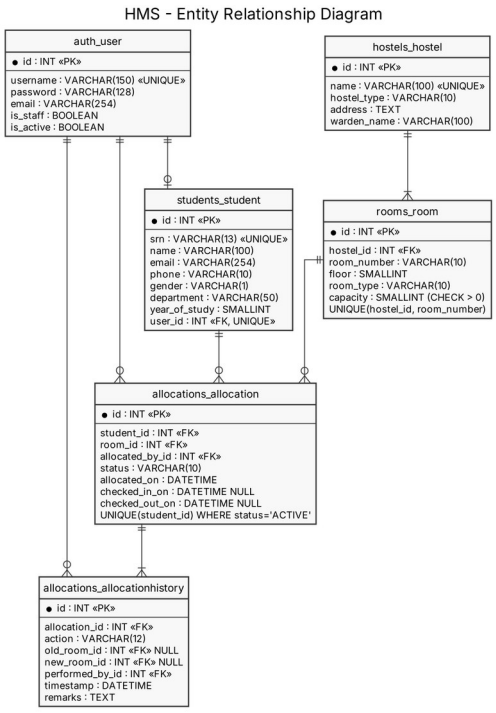


Figure 4: HMS entity-relationship diagram

2.7 Risks & Mitigations

Risk	Impact	Mitigation
Two admins allocate the last bed of a room at the same time	Room gets more students than its capacity (breaks F-011)	Allocation runs inside transaction.atomic() with select_for_update() on the room and recounts active allocations before saving (Figure 5).
Displayed availability does not match real allocations	Admin allocates a full room or leaves beds empty	Availability is always computed from ACTIVE allocations (ADR-04), never stored.
A student opens another student's data by changing the URL	Privacy breach (SR-006)	Student views get the student from request.user, never from an ID in the URL.
Hostel or room deleted while students are in it	Orphan allocations, wrong reports	ForeignKeys use on_delete=PROTECT; the UI shows an error instead of deleting.
SQLite file lost or corrupted during development	Loss of test data before demo	Migrations and seed fixtures kept in Git; database copied before every demo.
Merge conflicts in a four person team	Integration bugs, lost time	One app per member, feature branches, every pull request reviewed by one other member and all tests run before merge.
Scope creep (fees, visitors, notifications)	Core features not finished on time	Stick to the SRS; extra ideas go to section 3.6 as future work.

2.8 Traceability to Requirements

This table fills in the 'Design Spec' column of the SRS RTM (which was marked DS-TBD).

Requirement	Architecture element	Design element	Planned code location
HMS-F-001 Authentication	Authentication Module	Django LoginView, /accounts/login/	accounts/views.py
HMS-F-002 Role-based access	Authentication Module	@login_required, @admin_required (is_staff)	accounts/ decorators.py
HMS-F-003 Student registration	Student Module	Student model, StudentForm, /students/new/	students/models.py, forms.py
HMS-F-004 Update student	Student Module	/students/<id>/edit/	students/views.py
HMS-F-005 View own profile	Student Module	/student/home/ using request.user	students/views.py
HMS-F-006 Hostel management	Hostel Module	Hostel model, /hostels/ CRUD views	hostels/
HMS-F-007 Room management	Room Module	Room model with capacity constraint, /rooms/ views	rooms/models.py, views.py
HMS-F-008 Room availability	Room Module	Room.available_capacity(), /rooms/availability/, JSON API	rooms/models.py, api.py
HMS-F-009 View room details	Room Module	/rooms/<id>/	rooms/views.py
HMS-F-010 Room allocation	Allocation Module	allocate_room(), /allocations/new/, Figure 5	allocations/ services.py
HMS-F-011 Prevent over-allocation	Allocation + Room Module	Atomic capacity check, RoomFullError	allocations/ services.py
HMS-F-012 View allocation status	Allocation Module	Student.current_allocation(), /student/home/	students/models.py
HMS-F-013 Modify allocation	Allocation Module	modify_allocation(), /allocations/<id>/modify/	allocations/ services.py
HMS-F-014 Cancel allocation	Allocation Module	cancel_allocation(), /allocations/<id>/cancel/	allocations/ services.py
HMS-F-015 Check-in	Check-In Module	check_in(), /allocations/<id>/checkin/	allocations/ services.py
HMS-F-016 Check-out	Check-Out Module	check_out(), /allocations/<id>/checkout/, Figure 6	allocations/ services.py
HMS-F-017 Search student	Student Module	search_students(q), /students/?q=	students/services.py
HMS-F-018 Allocation history	Allocation Module	AllocationHistory model, /allocations/history/	allocations/ models.py
HMS-F-019 Admin summary	Dashboard Module	get_summary(), /dashboard/	dashboard/ services.py
HMS-NF-001 Performance	All modules	select_related(), annotate() counts, pagination	views, querysets
HMS-NF-002 Reliability	ORM / Database	Atomic transactions, constraints, PROTECT	models, services
HMS-NF-003 Security	Authentication Module	Section 2.9	accounts/, settings.py
HMS-NF-004 Usability	UI Client	Section 3.5	templates/
HMS-NF-005 Maintainability	Application structure	One Django app per module (ADR-01)	project layout
HMS-NF-006 Data integrity	Room + Allocation Modules	Computed availability (ADR-04), unique ACTIVE allocation	rooms/, allocations/
HMS-SR-001 to SR-006	Security architecture	Section 2.9 controls table	accounts/, settings.py, views

2.9 Security Architecture

Threat Modeling (STRIDE):

Threat	Example in HMS	Mitigation
Spoofing	Someone logs in as an admin using a guessed password.	Django auth with hashed passwords, password validators (minimum length, not common), login required on all pages.
Tampering	A forged form submission changes a student's room.	CSRF token on every POST form, server side form validation, only admins can POST to allocation URLs.
Repudiation	An admin says they never cancelled an allocation.	AllocationHistory stores action, performed_by and timestamp for every change.
Information disclosure	A student opens another student's profile URL.	Student pages read data only for request.user; DEBUG=False in deployment; HTTPS when deployed.
Denial of service	Very large lists or repeated login attempts slow the server.	Pagination on lists; optional login attempt limiting (for example django-axes) if deployed.
Elevation of privilege	A student account tries to open /students/new/.	@admin_required checks is_staff and returns 403; students are never created as staff.

Security requirement to control mapping:

Req ID	Control
HMS-SR-001	LOGIN_URL set; @login_required on every view except login.
HMS-SR-002	@admin_required decorator (is_staff) on all management views, plus a template check that hides admin menus from students.
HMS-SR-003	Django's default PBKDF2 password hasher; passwords never shown or logged.
HMS-SR-004	Django ModelForms with field validation (email format, phone digits, capacity > 0, unique SRN).
HMS-SR-005	Django session framework; session cleared on logout; SESSION_COOKIE_HTTPONLY and SESSION_COOKIE_SECURE (on HTTPS).
HMS-SR-006	Student views look up Student by request.user only; any other access returns 403/404.

3. Design

3.1 Design Overview

Every request follows the same path through the layers:

```
Browser -> urls.py -> View (login + role check) -> Form validation
        -> Service function / model method (business rules)
        -> Django ORM -> Database
        -> Template rendered and returned (or redirect with message)
```

Planned project structure (one Django app per component in Figure 1):

```
hms/                # project settings, root urls.py
accounts/           # login, logout, admin_required decorator
```

```
students/            # Student model, forms, views, tests
hostels/             # Hostel model, views, tests
rooms/               # Room model, available_capacity(), views, tests
allocations/         # Allocation, AllocationHistory, services.py
                     # allocate/modify/cancel/checkin/checkout views
dashboard/           # summary view
templates/  static/  # shared base.html, Bootstrap, CSS
```

Key business rules and where they live:

- A room's available capacity = capacity minus number of ACTIVE allocations (Room.available_capacity()).
- An allocation is only created if available capacity > 0 and the student has no other ACTIVE allocation (allocations/services.py, allocate_room()).
- Modify moves an ACTIVE allocation to a new room only if the new room has a free bed; history stores old and new room.
- Check-in is only allowed on an ACTIVE allocation that is not already checked in.
- Check-out is only allowed after check-in; it sets status to COMPLETED, which frees the bed.
- Cancel sets status to CANCELLED, which also frees the bed.

Allocation life cycle (state view):

```
allocate          -> ACTIVE (not checked in)
check_in          -> ACTIVE (checked in)
check_out         -> COMPLETED   (bed released)
cancel (any time while ACTIVE) -> CANCELLED   (bed released)
modify            -> stays ACTIVE, only the room changes (logged as MODIFIED)
```

Design rules: views call services, services call models and other modules' public services (never another app's views); every change to an allocation goes through allocations/services.py so the history log cannot be skipped.

3.2 UML Sequence Diagrams

Two flows are shown. Diagram 1 is the most important one because it shows how over-allocation is prevented.

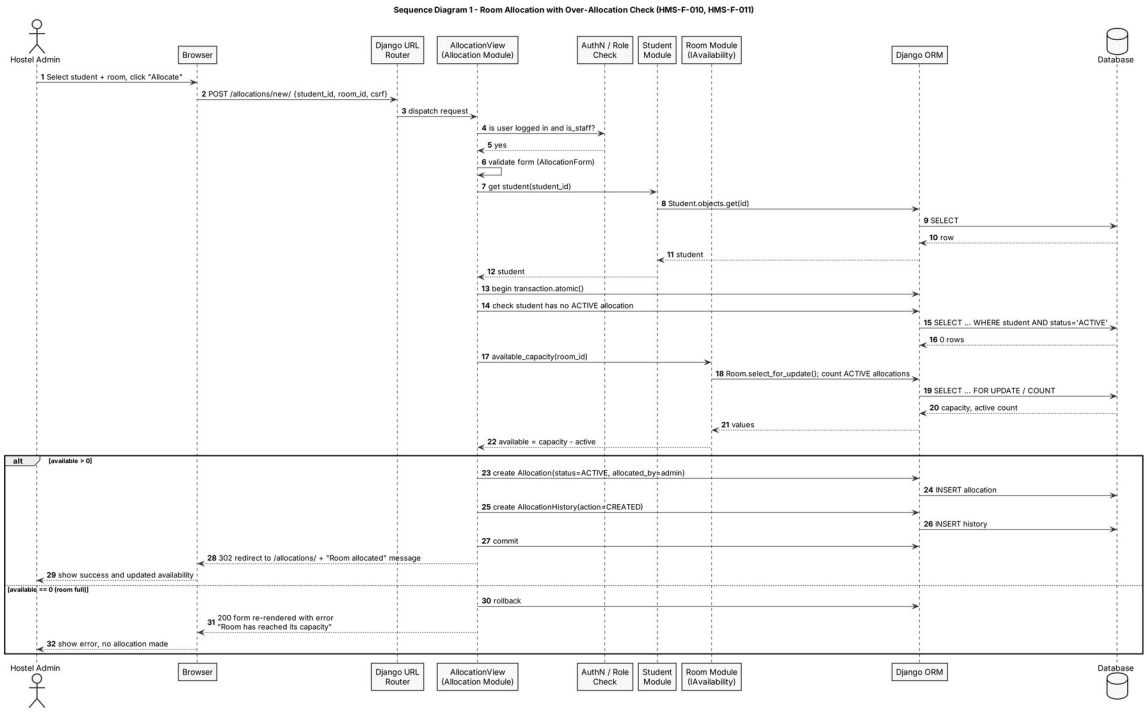


Figure 5: Sequence Diagram 1: Room allocation with over-allocation check (HMS-F-010, HMS-F-011)

In Diagram 1 the capacity check and the insert happen inside one database transaction, with the room row locked. So if two admins try to give away the last bed at the same moment, the second one sees the updated count and gets the 'Room has reached its capacity' error instead of creating a second allocation.

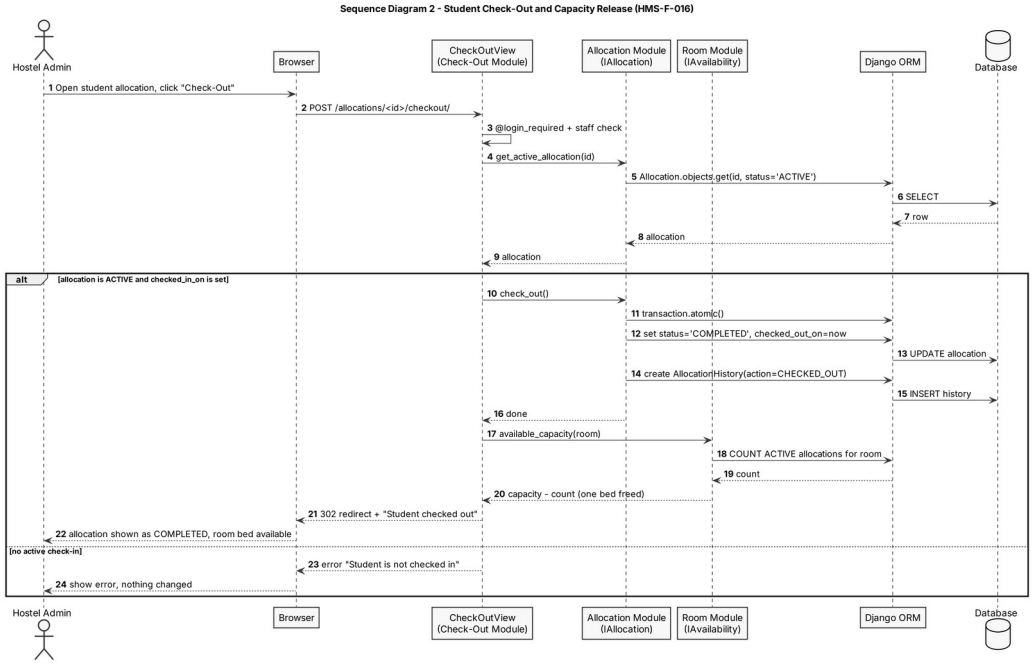


Figure 6: Sequence Diagram 2: Student check-out and capacity release (HMS-F-016)

In Diagram 2 nothing is deleted. The allocation status changes to COMPLETED, a history row is written, and since availability is computed from ACTIVE allocations the bed becomes free automatically.

3.3 API Design

HMS is a server-rendered Django app, so most of its interface is URL routes called by HTML forms (GET and POST). One small JSON endpoint is provided for room availability. All routes except login need a logged-in user, all POST forms need a CSRF token, and errors are shown as form or page messages.

3.3.1 Authentication Module: Login

Field	Value
Endpoint	/accounts/login/
Method	POST
Allowed role(s)	Public
Request	username, password, csrfmiddlewaretoken (form fields)
Response	302 redirect to /dashboard/ (admin) or /student/home/ (student); session cookie set
Errors	200 login page re-shown with 'Invalid username or password'; 403 missing CSRF token

3.3.2 Student Module: Register student

Field	Value
Endpoint	/students/new/
Method	POST
Allowed role(s)	Admin
Request	srn, name, email, phone, gender, department, year_of_study, csrfmiddlewaretoken
Response	302 redirect to /students/ with message 'Student registered'
Errors	200 form with field errors ('SRN already exists', 'Enter a valid email', 'Phone must be 10 digits'); 302 to login if not logged in; 403 not admin

3.3.3 Room Module: Room availability (JSON)

Field	Value
Endpoint	/api/rooms/<room_id>/availability/
Method	GET
Allowed role(s)	Admin, Student
Request	-
Response	200 {"room_id": 12, "hostel": "Block A", "room_number": "A-101", "capacity": 3, "active_allocations": 2, "available": 1}
Errors	302 to login if not logged in; 404 room not found

3.3.4 Allocation Module: Allocate room

Field	Value
Endpoint	/allocations/new/
Method	POST
Allowed role(s)	Admin
Request	student_id, room_id, csrfmiddlewaretoken
Response	302 redirect to /allocations/ with message 'Room allocated successfully'; Allocation (ACTIVE) and history row CREATED saved
Errors	200 form error 'Room has reached its capacity' (RoomFullError); 200 form error 'Student already has an active allocation' (AlreadyAllocatedError); 403 not admin; 404 student or room not found

3.3.5 Allocation Module: Modify / cancel allocation

Field	Value
Endpoint	/allocations/<id>/modify/ and /allocations/<id>/cancel/
Method	POST
Allowed role(s)	Admin
Request	modify: new_room_id, csrfmiddlewaretoken; cancel: remarks (optional), csrfmiddlewaretoken
Response	302 redirect to allocation detail with message; history row MODIFIED (old and new room) or CANCELLED
Errors	error message 'Room has reached its capacity' (modify to full room); 'Allocation is not active' (InvalidStateError); 403; 404

3.3.6 Check-Out Module: Check out student

Field	Value
Endpoint	/allocations/<id>/checkout/
Method	POST
Allowed role(s)	Admin
Request	csrfmiddlewaretoken
Response	302 redirect to allocation detail with message 'Student checked out'; status COMPLETED, checked_out_on set, bed freed
Errors	error message 'Student is not checked in' or 'Allocation is not active' (InvalidStateError); 403 not admin; 404 allocation not found

Internal service interface of the Allocation Module (allocations/services.py), used by the views above:

```
allocate_room(student, room, by) -> Allocation
    raises RoomFullError, AlreadyAllocatedError
modify_allocation(allocation, new_room, by) -> Allocation
    raises RoomFullError, InvalidStateError
cancel_allocation(allocation, by) -> None       raises InvalidStateError
check_in(allocation, by) -> None                raises InvalidStateError
check_out(allocation, by) -> None               raises InvalidStateError
```

3.4 Error Handling, Logging & Monitoring

- **Validation errors:** shown next to the field using Django forms (for example 'SRN already exists', 'Capacity must be at least 1').
- **Business rule errors:** custom exceptions (RoomFullError, AlreadyAllocatedError, InvalidStateError) are caught in the view and shown as a red message using Django's messages framework. The transaction is rolled back so no partial data is saved.
- **Success messages:** green message after every successful create, update, allocate, check-in or check-out.
- **Error pages:** custom 403, 404 and 500 templates so users never see a Python stack trace; DEBUG=False in deployment.
- **Logging:** Python logging module writing to logs/hms.log. INFO for allocation, cancel, check-in and check-out events; WARNING for failed logins and permission denied; ERROR for unexpected exceptions. Passwords and personal details (phone, email) are never logged, only IDs.
- **Monitoring:** for this project, monitoring is simple: the admin dashboard shows occupancy, the log file is checked for errors, and response times are measured during performance testing (TC-PERF-01).

3.5 UX Design

Main screens:

Screen	User	Notes
Login	All	Username and password; clear error on wrong credentials.
Admin Dashboard	Admin	Cards for total students, rooms, total beds, occupied and free beds; per hostel table.
Students list / search / form	Admin	Search box (SRN or name), paginated table, Add and Edit buttons.
Hostels and Rooms	Admin	Room table with capacity, occupied and available; availability shown as text plus colour badge (Full / Available).
Allocate Room	Admin	Pick student (only students without an active allocation) and room (only rooms with free beds).
Allocation Detail	Admin	Shows status and dates, with Check-in, Check-out, Modify and Cancel buttons enabled only when allowed.
Allocation History	Admin	Filter by student or room; shows action, who and when.
Student Home	Student	Profile details and an allocation card (hostel, room, status) or 'No room allocated yet'.

Design principles:

- Same navigation bar on every page, with menu items based on role (students never see admin links).
- Clear labels and form validation messages in plain English (HMS-NF-004).
- Confirmation step before destructive actions like Cancel and Check-out.
- Status is shown with text as well as colour, so it is readable for colour-blind users.

- Bootstrap layout so pages also work on mobile screens; all forms usable with keyboard only.

3.6 Open Issues & Next Steps

Open issues:

- Allocation is tracked at room level (capacity), not at individual bed level. If the hostel needs bed numbers, a Bed model can be added later.
- Should allocation check that the student's gender matches the hostel type? Currently left to the admin.
- Final deployment database (SQLite or PostgreSQL) to be decided with the course coordinator.
- Password reset by email is not included in this version; admins reset passwords from Django admin.

Next steps:

- Implement models and migrations, then the allocations service functions with unit tests.
- Build views and templates module by module and run tests as per the STP.
- Update the RTM status column from N to P/A as features are implemented and tested.

Future enhancements: online fee payment, visitor management, email or SMS notifications, students requesting rooms online.